AI COMPILER

Discovering missed compiler optimizations

LLM-guided experiments for AscendNPU-IR

Experimental project proposal

16 October 2026



What are we trying to do?

Find optimization gaps with less compiler-engineer time.

Run candidate search and experiments in the background.
Bring the AscendNPU-IR team reproducible findings to review.

Success means useful findings with less active investigation effort.

[REDO] How is this done today?

01 Locate expensive behavior

Connect a workload’s runtime cost to the relevant IR and generated code.

02 Establish a better alternative

Find a legal rewrite and determine whether its benefit survives compilation.

03 Prepare a compiler finding

Package the conditions, reproducer and measurements for developer review.

Typical workflow to confirm with the compiler team.

Limits of current methods

Method	Limit relevant to this project
Profiling and inspection	Engineers connect runtime cost to a legal change.
Performance regression tests	Covered baselines can miss unrealized improvements.
Automated transformation search	Semantic coverage and cost models constrain the search.

The team’s actual investigation effort and pain points still need measurement.

What are we proposing?

01 Connect existing compiler and NPU tools

Adapters capture IR, compile variants, run output checks and collect measurements.

02 Offload the search

A controller runs bounded LLM experiments, retains feedback and stops poor searches.

03 Return evidence for compiler work

Reproducible cases with static evidence, optional timings and reviewable assumptions.

Why might this work?

	LPO	PRAGMA
Task	Find replacement sequences in LLVM IR	Optimize generated kernels using profiling feedback
Reported result	41 confirmed or fixed missed optimizations	2.81× CPU and 2.30× GPU average speedup vs Torch
Important limit	No significant SPEC runtime gain for selected patches	Kernel results do not establish compiler fixes or NPU gains

Published results motivate an experiment. Ascend effectiveness remains untested.

Sources: LPO, arxiv.org/abs/2508.16125 • PRAGMA, arxiv.org/abs/2511.06345

The hypothesis we will test

01 Useful alternatives

The model exposes missed simplifications or resource reductions after compilation.

02 Evidence guides further attempts

Diagnostics, static costs and optional runtime evidence guide further attempts.

03 Less active engineering work

Engineers spend less time preparing experiments and more time reviewing useful cases.

Background execution does not automatically reduce elapsed time or device use.

Who benefits if we succeed?

01 AscendNPU-IR optimization developers

Receive reproducible candidates for missing rewrites, pass interactions or missed triggers.

02 AscendNPU-IR performance engineers

Offload repeated proposals and runs; inspect a filtered set of supported findings.

03 Users of affected Ascend workloads

Benefit if the team implements and deploys the resulting compiler improvements.

Other compilers and general-purpose adoption are outside this experiment.

[REDO] What could go wrong?

Risk	Response
Faster but incorrect code	Limit semantics and keep proof/test evidence distinct.
Assumptions the compiler lacks	Check every precondition at the chosen boundary.
Noise or gains that vanish in context	Reinsert the change and confirm on fresh runs.
Invalid proposals	Start small; use compilation and output checks.
Too few useful findings for the effort	Compare yield, device time and reviewer effort.

A running search loop alone does not establish practical value.

Initial scope and limits

One IR boundary. One hardware configuration.

Candidate family	Example opportunity
Index and shape computation	Repeated arithmetic, address calculations or redundant checks
Data movement	Avoidable copies or materialization with explicit memory semantics

Choose one family first. Use static shapes and agreed numerical semantics.
Reserve separate workload families for evaluation.

How long could it take?

One experienced engineer with part-time compiler-team support.

Outcome	Elapsed time from project start
Narrow prototype for one family	2–3 months
Evaluated pilot, including the prototype	4–6 months total

Provisional estimates. They assume existing execution and checking infrastructure.
Re-estimate after the first stage.

Ordered implementation stages

Stage	Work	Completion evidence
1. Boundary	Capture, replace and replay IR	Original and manual variant execute
2. Harness	Static scores, outputs and corpus	Replayable evidence and check limits
3. Check contract	Inputs, tolerances and controls	Seeded mismatches are rejected
4. Search	LLM, feedback and experiment store	Bounded unattended attempts
5. Evaluation	Held-out cases and team review	Useful findings and active time costs

Research question: which findings need NPU timing, and which can be reported earlier?

[REDO] Progress methodology

01 Define the evaluation cases

Separate known controls from unseen cases; decide what counts as one useful finding.

02 Make the comparison fair

Match tasks and checking rules; account for operator familiarity and search variability.

03 Measure the work that is offloaded

Log setup, interventions and review time alongside finding yield and device use.

What would count as success?

Claim	Evidence needed
Can discover useful gaps	A new AscendNPU-IR case with agreed correctness and improvement evidence, accepted by the team.
Can save engineer time	Comparable useful findings with less active setup, intervention and review time than the agreed baseline.

Report the time spent reviewing false positives and unsuccessful searches.

Static evidence can support a report; a runtime-gain claim needs execution timing.

How to test engineer-time savings

Measure	Current workflow	With the framework
Engineer effort	Prepare, run, diagnose and package experiments	Configure, intervene and review findings
Useful outcome	Distinct cases accepted by the AscendNPU-IR team	Same acceptance criteria
Other costs	Elapsed time and NPU consumption	Elapsed time, NPU use and model consumption

Compare useful findings per active engineer-hour; include failed attempts.

Research foundations

LPO

Discovering Missed Peephole Optimizations with Large Language Models

arxiv.org/abs/2508.16125 • Compiler-gap discovery and checked rewrites

PRAGMA

A Profiling-Reasoned Multi-Agent Framework for Automatic Kernel Optimization

arxiv.org/abs/2511.06345 • Profiling feedback and execution checks

[REDO] System architecture

Layer	Components	Responsibility
Customer adapters	IR, compiler, cost metrics, reference and runtime	Expose the target and its semantic contract
Experiment core	Case selection, LLM interface, validation and search control	Propose, check and compare bounded experiments
Evidence and review	History, minimization, reports and triage	Preserve reproducibility and classify useful findings

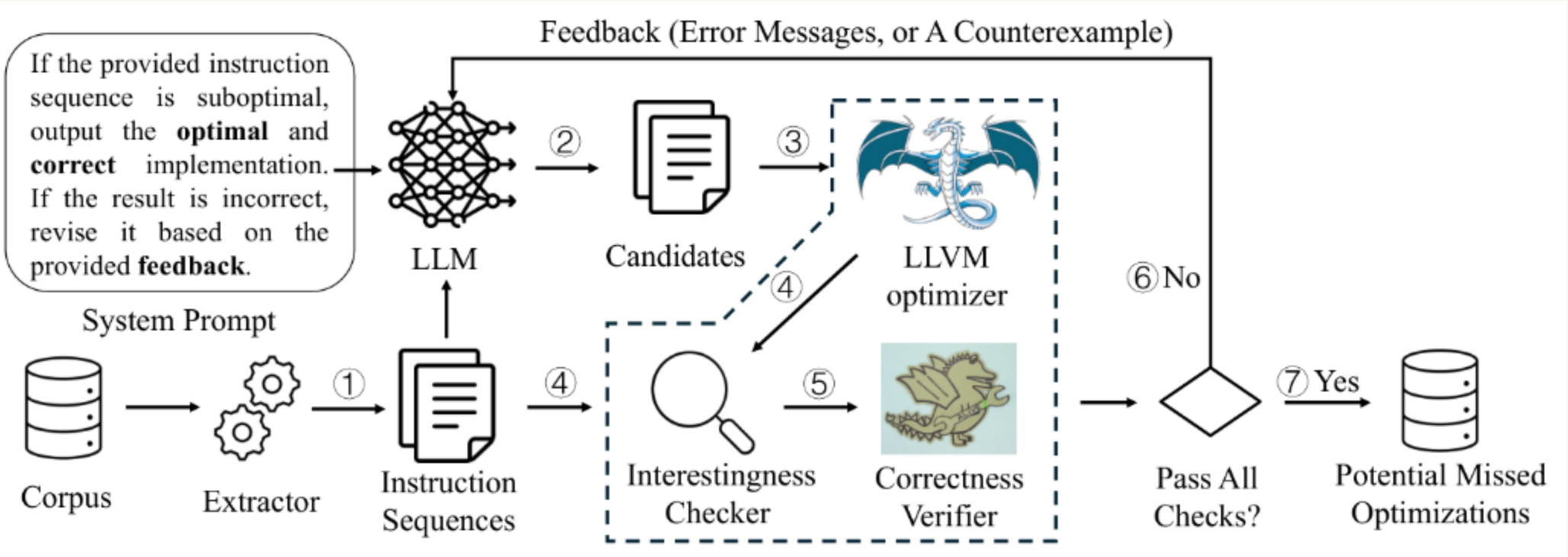
The controller owns acceptance rules. The LLM proposes code and hypotheses.

[REDO] Ascend integration points

Observed starting point	Work still required
HFusion and HIVM pipelines	Choose a stable IR boundary and replay path.
HIVM vector-add integration	Generalize compilation, arguments and device execution.
Documented msprof workflow	Check metric coverage and code-to-profile mapping.
Documented mssanitizer workflow	Check launcher support and instrumented behaviors.

Source evidence only. Tool availability and end-to-end execution still need testing.

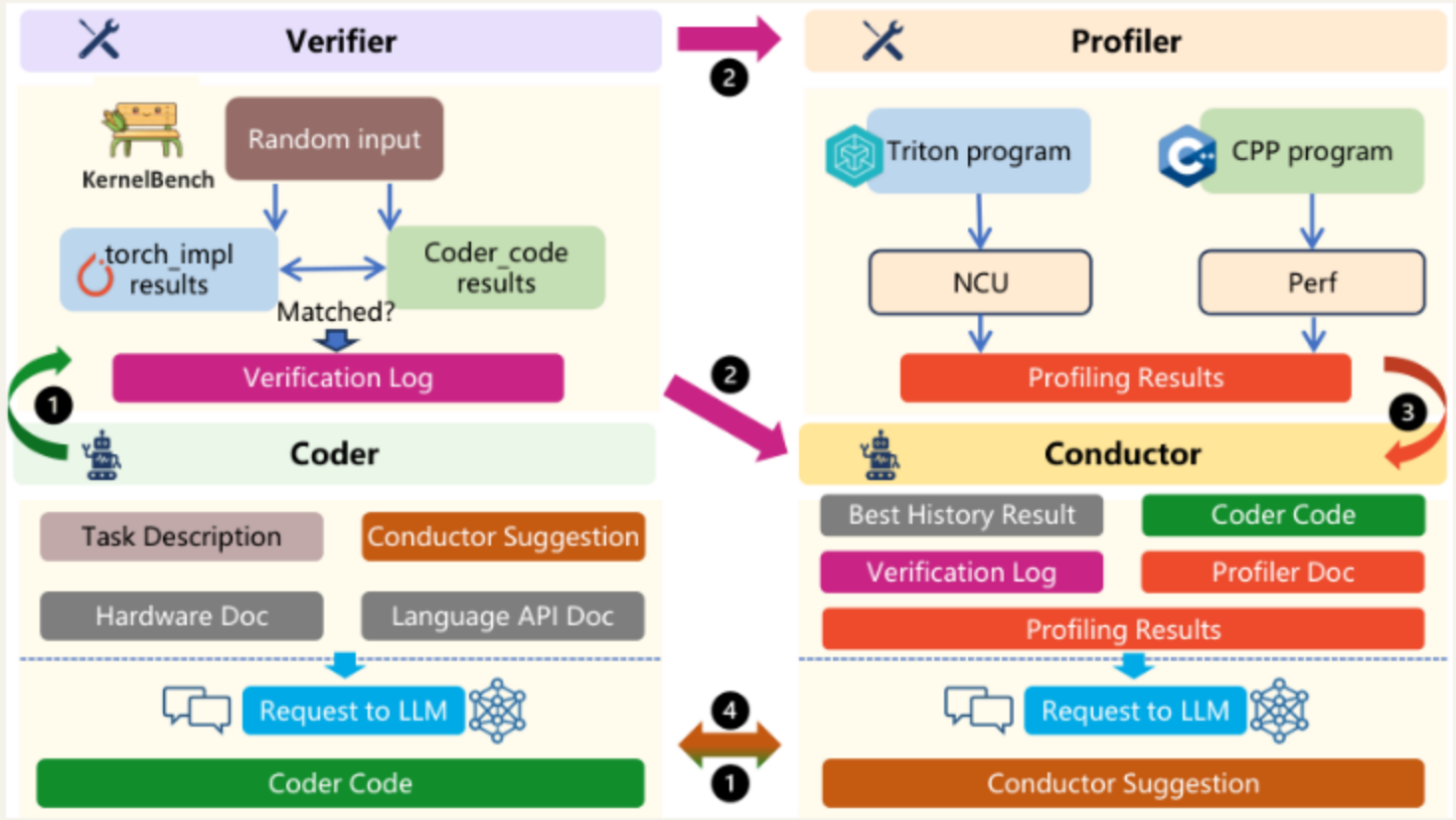
LPO: finding compiler gaps



Reuse: static screening before expensive checks. Target execution is not required by LPO.

Xu et al., LPO, Figure 2 • arxiv.org/abs/2508.16125

PRAGMA: using performance feedback



Reuse

Output comparisons
Profiles with context
Variant history

Adapt

Ascend measurements
Compiler-gap attribution

PRAGMA, Figure 1 • arxiv.org/abs/2511.06345

[REDO] Correctness and attribution

Preserve behavior

Types, shapes and layouts

Aliasing, effects and lifetimes

Integer and floating-point semantics

Establish compiler relevance

Invariants available at the IR boundary

Same compiler options and suffix

Benefit retained in the original operator

Start with output checks. General correctness remains a later review obligation.

Start with compilation and outputs

First iteration: compile, run and preserve the results we care about.

Fix the outputs, test inputs and numerical tolerances before search.
Reject compilation failures, crashes and observed mismatches.

Optional later tool	Potential use; coverage must be checked
MLIR-TV	Selected standard tensor operations
xdsl-smt	Small arithmetic subsets or explicit custom semantics
TensorRight	Generalized tensor rewrite rules in its own DSL

Passing tests supports a candidate finding; it does not prove equivalence for all inputs.

Why Alive2 appears in this proposal

01 **LPO needs an independent correctness check**

Alive2 checks whether a replacement refines the original LLVM IR.

02 **AscendNPU-IR has a different semantic surface**

HFusion/HIVM device operations are not automatically covered by Alive2.

03 **A possible later extension**

Use a justified LLVM export for a supported subset; begin with execution checks.

Sources: LPO, arxiv.org/abs/2508.16125 • github.com/AliveToolkit/alive2

[REDO] Execution and proof tooling

Mechanism	Role and limitation
MLIR SMT dialect	Build solver queries. Target semantics still need encoding.
Reference execution	Compare outputs for tested inputs under an agreed contract.
IR verifier and sanitizer	Check structure and supported runtime fault conditions.
Validation record	Separate tested, proved, failed, unsupported and timed-out cases.

First iteration uses compilation and output checks; other tooling remains optional.

[RESEARCH] Must the metric be time?

Evidence	Cost and claim
Static code / resource changes	No target execution for scoring. Shows simplification or reduced allocation; speedup remains unknown.
Target cost estimate	Needs a credible NPU cost model. Predicts benefit; model coverage and ranking need evaluation.
Execution latency	Needs runnable cases, device access and repeated measurements. Confirms speed for tested conditions.

Can static reports save resources while retaining value for the compiler team?

LPO: arxiv.org/abs/2508.16125 • Correctness checks are a separate requirement.